

Orli - A new take on Print-on-Demand

Project Team

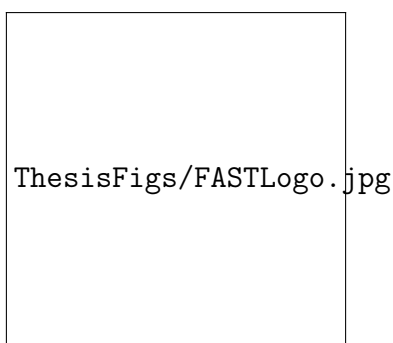
Aisha Adil	22I-2718
Muhammad Uzair Choudhary	22I-2681
Anishra Ashfaq	22I-2614

Session 2022-2026

Supervised by

Dr. Behjat Zuhaira

Co-Supervised by



Department of Software Engineering

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction	1
1.1	Existing Solutions	2
1.2	Problem Statement	3
1.3	Scope	3
1.4	Modules	4
1.4.1	Module 01: Client Web Application (React Frontend)	4
1.4.1.1	Customer Interface	5
1.4.1.2	Designer Interface	5
1.4.1.3	Manufacturer Interface	5
1.4.2	Module 02: Admin Web Application (React + Node.js)	6
1.4.2.1	Admin Interface	6
1.4.3	Module 03: AI Services (Flask + TensorFlow)	6
1.4.3.1	AI and Creative Intelligence Module	6
1.4.4	Module 04: Data Management Services (MongoDB-tentative)	6
1.4.4.1	Data Management Module	7
1.4.5	Module 05: Core Server Backend and Financial Operations (Node.js + Express + Payment Gateway)	7
1.4.5.1	Core Business Logic Module	7
1.4.5.2	Financial Operations Module	7
1.5	Work Division	8
2	Project Requirements	9
2.1	Use-case	9
2.2	Functional Requirements	15
2.2.1	Module 1: Client Web Application (React Frontend)	15
2.2.1.1	Customer Interface	15
2.2.1.2	Designer Interface	17
2.2.1.3	Manufacturer Interface	18
2.2.1.4	Admin Interface	19
2.2.2	Module 2: AI and Creative Intelligence	20
2.2.3	Module 3: Core Business Logic (Node.js + Express)	21
2.2.4	Module 4: Data Management (MongoDB)	22

2.2.5	Module 5: Financial Operations (Payment Gateway Integration)	22
2.3	Non-Functional Requirements	23
2.3.1	Reliability	23
2.3.2	Usability	23
2.3.3	Performance	23
2.3.4	Security	23
3	System Overview	25
3.1	Architectural Design	25
3.2	Data Design	28
3.2.0.1	Data Transformation	28
3.2.0.2	Major Entities	28
3.2.0.3	Data Storage Systems	28
3.2.0.4	Processing and Organization	28
3.3	Data Design	29
3.3.1	Transformation of Information Domain	29
3.3.2	Major System Entities	29
3.3.3	Data Storage and Organization	29
3.4	Domain Model	30
3.5	Design Models	32
4	Implementation and Testing	39
4.1	Algorithm Design	39
4.2	External APIs/SDKs	39
	References	43

List of Figures

3.1	System Architecture	26
3.2	System Architecture - Box and Line Diagram	27
3.3	Domain Model	31
3.4	Class Digram	32
3.5	DEsigner payout SD	32
3.6	DEsigner payout SD	33
3.7	ERD	33
3.8	Update Inventory - SD	34
3.9	Withdraw Earnings - SD	34
3.10	Manage Inventory - SSD	35
3.11	System Activity Diagram	36
3.12	Activity for Manufacturer	37
3.13	Activity for Designer	38

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division	8
2.1	Use Case 1: Browse and Search Products	10
2.2	Use Case 2: Place and Pay for Order	11
2.3	Use Case 3: Create and Enhance Design (AI)	12
2.4	Use Case 4: Manage Storefront	12
2.5	Use Case 5: Fulfill Order	13
2.6	Use Case 6: Moderate Platform Content	14
4.1	Third-Party APIs and SDKs	41

Chapter 1

Introduction

This scope document outlines the proposed development of a digital platform designed to empower local artists by connecting them with global markets through stream-lined, AI-enhanced e-commerce experience. The purpose of this document is to define our vision, the boundaries, and the technical direction of the project, ensuring clarity for all stakeholders and establishing a shared understanding of what the system will—and will not—deliver.

The primary aim of this document is to present a clear and actionable road-map for building said web-based platform that may serve as a foundational reference for the project team, academic evaluators, and potential collaborators. It may detail the system's intended features, technological stack, development phases, and limitations. By articulating the scope now, we aim to prevent scope creep, aligning expectations, and guide decision-making throughout the life cycle of the project.

In today's digital economy, platforms like Shopify, Etsy, WooCommerce, Printify, and Gelato have revolutionized how creators monetize their work. However, these systems often cater to established businesses or lack localized support for emerging artists or business-minded individuals. This project seeks to bridge that gap by offering a tailored solution for student creators, independent designers, and amateurs, particularly those with limited resources. The envisioned platform will be built using the MERN stack—MongoDB, Express.js, React, and Node.js—providing a robust and scalable foundation for web development. AI modules, developed in Python, will offer intelligent design assistance and personalization.

This document sets the stage for a collaborative, well-defined development process—one that celebrates creativity, leverages technology, and delivers real-world impact for emerging artists and entrepreneurs.

1.1 Existing Solutions

The current landscape of the digital economy has been significantly shaped by platforms such as Shopify, Etsy, WooCommerce, Printify, and Gelato, which have revolutionized the way creators monetize their work. These global leaders function primarily as e-commerce engines or aggregators that connect merchants with print providers. However, a critical assessment reveals that these systems often cater to established businesses and lack localized support for emerging artists, students, or business-minded individuals in developing markets like Pakistan. Furthermore, while these platforms excel in logistics and storefront management, they fall short in offering advanced, integrated AI-powered design tools within the platform itself. This technological and infrastructural gap results in inefficiencies in production, higher costs, and slower delivery times for local consumers. The proposed solution, Orli, aims to bridge these gaps by offering a localized, AI-enhanced ecosystem that integrates design generation, manufacturing, and sales into a unified platform.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Printify / Gelato	Global Print-on-Demand (POD) aggregators that connect merchants with a network of international print providers.	These platforms are not optimized for local (Pakistani) infrastructure, leading to high shipping costs and production inefficiencies. They generally lack built-in AI generative tools for design creation.
Shopify / WooCommerce	Robust e-commerce frameworks that allow users to build scalable online stores and manage business operations.	They primarily cater to established businesses rather than amateurs or students with limited resources. They require third-party plugins for POD integration and do not provide native AI design assistance.
Etsy	A global marketplace focused on handmade, vintage, and creative goods, allowing independent artists to list products.	It lacks an integrated, localized manufacturing network for Pakistan, forcing artists to handle their own fulfillment. It offers limited access to unique, culturally rich designs for local markets.

1.2 Problem Statement

In the evolving landscape of the print-on-demand industry, there is a notable absence of an integrated platform that bridges the gap between local talent and global markets. Existing solutions fall short in offering advanced AI-powered design tools and a marketplace that caters to both local and international demands. This gap creates challenges for local artists to showcase their work on a global stage and limits customers' access to unique and culturally rich designs. It's a missed opportunity for both creators and consumers. Expanding further, this gap limits the growth of independent designers and weakens the customer experience. Our platform addresses this by introducing a localized POD model tailored for emerging artists and entrepreneurs. Beyond simplifying production and fulfillment, it empowers creators to build and manage their own brands, laying the foundation for sustainable digital entrepreneurship in Pakistan.

1.3 Scope

The scope of this project encompasses the development of a digital platform named Orli, designed to bridge the gap between local manufacturers, creative designers, and global consumers. The system will be constructed using the MERN stack (MongoDB, Express.js, React, and Node.js) to provide a robust and scalable foundation for web development. Key functionality includes the creation of personalized user profiles where designers can upload original artwork or utilize integrated tools for content creation. The project will feature an advanced AI module featuring fine-tunings, specifically designed to offer design generation, image enhancement, and context-aware mock-ups. Users will be provided with real-time customization previews, enabling them to visualize their designs on various merchandise formats such as t-shirts and stickers. To facilitate logistics, the platform will include a dedicated manufacturer dashboard for order tracking and production coordination. The scope also covers the development of a curated marketplace equipped with SEO-optimized e-commerce capabilities to enhance product discoverability. Furthermore, an integrated analytics dashboard will be built to provide stakeholders with insights into sales performance and customer engagement. Financial operations are included within the scope, specifically focusing on automated split payments and transparent revenue sharing. An administrative interface will also be delivered to handle user management, system monitoring, and content moderation.

Despite these extensive features, specific boundaries have been established to ensure the project remains feasible within the given timeline. The scope explicitly excludes the development of custom plugins for external platforms. Advanced marketing automation tools are also outside the boundaries of this project. Additionally, the system will not

support complex enterprise-level integrations at this stage. The project focuses strictly on software development and does not include the management of physical printing hardware or shipping logistics beyond the software interface. By maintaining these limitations, the project aims to deliver a focused, technically sound solution aligned with the team's current resources.

1.4 Modules

In its most rudimentary form, our app is an e-commerce platform. But on a closer look, it's a web-application that re-imagines the traditional online marketplace by merging the core functionalities of platforms like Printify, Shopify, and Etsy into a unified e-commerce ecosystem. Unlike conventional e-commerce models in Pakistan, our platform serves as a dynamic, multi-sided marketplace that connects three key stakeholders: manufacturers, independent designers, and end consumers.

At its core, the platform empowers designers to build their own brands and storefronts by sourcing materials directly from verified print providers. Designers can create and showcase custom products using integrated design tools, including Adobe-style or Canva-style interfaces and AI-powered modeling features. These tools allow for real-time 3D-rendering enabling designers and customers to visualize products in their environment before publishing/purchasing.

To ensure clarity in architecture and role-based functionality, the system is divided into five major modules.

1. Client Web Application
2. Admin Web Application
3. AI Services (Intelligent Assistance Layer)
4. Data Management Services
5. Core Server Backend & Financial Operations

Each system type contains relevant modules and sub-modules that interact through RESTful APIs and shared data pipelines. Below is the architectural visualization of our system focusing on each module separately to ensure all the details are up-front.

1.4.1 Module 01: Client Web Application (React Frontend)

This module represents all user-facing interactions, primarily designed using **React.js** for responsive, modular, and component-driven interfaces. It includes three major user

roles—**Customer**, **Designer**, and **Manufacturer**—each with its own dedicated UI module.

1.4.1.1 Customer Interface

It handles all front-end functionalities for end-users who browse, preview, and purchase products. Features include:

1. Preview Products
2. Checkout and Track Orders
3. View Trends
4. Browse Products

1.4.1.2 Designer Interface

This interface empowers designers to create, manage, and sell custom merchandise. Features include:

1. Design Canvas (Canva/Adobe-like interface)
2. Storefront Creation and Management
3. Order Management (may be changed on implementation)
4. Trend Analytics
5. Product Page SEO-Optimization

1.4.1.3 Manufacturer Interface

It caters to verified production/print partners who shall handle printing, inventory, and shipping. Features include:

1. Manage Orders (accept/reject/defer)
2. Print and Ship
3. Manage Inventory
4. Manage Performance Statistics

1.4.2 Module 02: Admin Web Application (React + Node.js)

This is a dedicated administrative dashboard providing complete system control, monitoring, and moderation. Its features are as follows:

1.4.2.1 Admin Interface

1. User Management
2. System Monitoring
3. Content Moderation
4. Analytics Dashboard

This system enables the administrators to manage platform-wide operations, maintain quality control, and track user and transaction data through integrated analytics.

1.4.3 Module 03: AI Services (Flask + TensorFlow)

This subsystem represents the **intelligent assistance layer**, responsible for creativity enhancement, visual rendering, and predictive design functionalities. These services are implemented using Python-based frameworks and can be deployed independently for scalability and GPU optimization.

1.4.3.1 AI and Creative Intelligence Module

This module provides AI-based functionalities to enhance design workflows and user creativity. It features include:

1. 2D Preview Generator
2. 3D Renderer
3. Design Suggestion/ Generation Engine
4. Generative Canvas Enhancer

1.4.4 Module 04: Data Management Services (MongoDB-tentative)

This subsystem is dedicated to storing, organizing, and securing all persistent data across the platform. It ensures high availability, schema integrity, and fast access to stored entities.

1.4.4.1 Data Management Module

This module is responsible for structured data storage, schema design, and secure data access.

1. Collection Creation
2. Collection Management
3. Security Enforcement
4. Data Interface and Configuration Management

1.4.5 Module 05: Core Server Backend and Financial Operations (Node.js + Express + Payment Gateway)

This is the **central orchestration layer** of the system that manages communication between frontends, AI services, and the database. It also handles all business logic, financial workflows, and authentication.

1.4.5.1 Core Business Logic Module

1. Product Service
2. Order Service
3. Payment Service
4. Authentication Service
5. Messaging Service
6. SEO Service

1.4.5.2 Financial Operations Module

Handles automated, secure, and transparent financial transactions.

1. Split Payments
2. Designer Payouts
3. Manufacturer Payouts
4. Premium Billing Plans

1.5 Work Division

For each module and respective feature, the responsibility of each team member is given in the following table as a decimal showing the module number and feature number(e.g. x.y shows x module and y feature, module 1 sub modules are considered features here):

Table 1.2: Work Division

Name	Registration	Responsibility/ Module/ Feature
Aisha Adil	22i-2718	Documentation, Designer interface, Generative engine, Data Management, Designer Backend
Muhammad Uzair Choudhary	22i-2681	Manufacturer Backend, 2d previewer, 3d renderer
Anishra Ashfaq	22i-2614	Design, Manufacturer Interface, Canvas Enhancer

This work division pertains to FYP I only.

Chapter 2

Project Requirements

This section outlines the necessary requirements for the successful completion of the project. Project requirements can be divided into two main categories: functional requirements, which describe the system's core operations, and non-functional requirements, which specify performance, security, and usability standards.

2.1 Use-case

This section describes how users will interact with the system through various scenarios, often referred to as use cases or event responses.

A use case represents a specific interaction between the user and the system, detailing the steps involved and the expected system responses. Each use case typically includes key components such as a unique identifier, a description of the interaction, the user or system actor involved, preconditions, the main flow of events, and postconditions. Orli's identified usecases include:

1. Browse/Search Products
2. Place Order
3. Manage Account (generic and for all users in every platform, therefore not considered)
4. Create Design
5. Enhance/Generate Design using AI
6. Manage Storefront
7. Moderate Platform (includes, users, content, etc.)

8. Monitor Analytics

The detailed use-cases are as follows:

Table 2.1: Use Case 1: Browse and Search Products

Field	Description
Use Case ID	UC-01
Use Case Name	Browse and Search Products
Primary Actor	Customer
Pre-Conditions	1. System is online and accessible. 2. Product catalog contains active listings.
Post-Conditions	1. Customer is presented with a list of products matching their criteria. 2. Search queries are logged for analytics.
Main Success Scenario	1. Customer navigates to the marketplace homepage. 2. Customer enters keywords into the search bar or selects a category. 3. System validates the input and retrieves matching products. 4. System displays results in a grid view with thumbnails, titles, and prices. 5. Customer applies filters (e.g., price range, category, designer). 6. System updates the displayed results dynamically. 7. Customer clicks on a specific product. 8. System displays the detailed product page (triggers UC-02 flow).
Alternate Flows	3a. Invalid Input: System sanitizes input and prompts user to re-enter. 4a. No Results Found: System displays a "No results found" message and suggests trending items. 11a. Product Unavailable: System displays an "Out of Stock" or "Archived" message.

Table 2.2: Use Case 2: Place and Pay for Order

Field	Description
Use Case ID	UC-02
Use Case Name	Place and Pay for Order
Primary Actor	Customer
Secondary Actors	Payment Gateway, Designer, Manufacturer
Pre-Conditions	1. Customer is logged in. 2. Customer has added items to the shopping cart.
Post-Conditions	1. Order is recorded in the system. 2. Payment is successfully processed. 3. Notifications are sent to the Designer and Manufacturer.
Main Success Scenario	1. Customer initiates the checkout process from the shopping cart. 2. System validates item availability and calculates total cost (including tax/shipping). 3. Customer provides or confirms shipping address. 4. Customer selects a payment method and enters credentials. 5. System transmits encrypted payment data to the External Payment Gateway. 6. Payment Gateway authorizes the transaction and returns a success token. 7. System records the order and updates inventory. 8. System sends an order confirmation to the Customer. 9. System notifies the Designer (of the sale) and Manufacturer (of the production task).
Alternate Flows	2a. Item Out of Stock: System alerts Customer and removes the item from the cart. 6a. Payment Declined: Payment Gateway returns an error. System prompts Customer to retry or use a different method. 6b. Gateway Timeout: System checks transaction status; if indeterminate, prompts Customer to contact support.

Table 2.3: Use Case 3: Create and Enhance Design (AI)

Field	Description
Use Case ID	UC-03
Use Case Name	Create and Enhance Design
Primary Actor	Designer
Secondary Actor	AI Service
Pre-Conditions	1. Designer is logged in. 2. Designer has accessed the Design Canvas tool.
Post-Conditions	1. New design asset is saved to the designer's library. 2. High-resolution file is ready for product mapping.
Main Success Scenario	1. Designer selects "Create New Design" on the dashboard. 2. System loads the design canvas interface. 3. Designer uploads a base image or enters a text prompt. 4. Designer selects an AI enhancement tool (e.g., Upscale, Style Transfer). 5. System sends the request to the AI Service. 6. AI Service processes the request and returns the enhanced image. 7. System displays the enhanced design for review. 8. Designer accepts the changes and clicks "Save". 9. System stores the design asset securely.
Alternate Flows	4a. Invalid File Format: System alerts Designer to upload a supported format (PNG/JPG). 6a. AI Service Failure: System displays an error message and restores the original state of the canvas.

Table 2.4: Use Case 4: Manage Storefront

Field	Description
Use Case ID	UC-04
Continued on next page	

Table 2.4 – continued from previous page

Field	Description
Use Case Name	Manage Storefront
Primary Actor	Designer
Pre-Conditions	1. Designer account is verified. 2. Designer has at least one saved design.
Post-Conditions	1. Storefront information is updated publicly. 2. Products are indexed for search.
Main Success Scenario	1. Designer navigates to the "Store Management" section. 2. Designer updates storefront details (Logo, Bio, Banner). 3. Designer selects designs to publish as products. 4. Designer sets pricing and inputs SEO metadata (Tags, Titles). 5. System validates the input fields (e.g., character limits, banned words). 6. Designer clicks "Publish". 7. System updates the public storefront and indexes products for search.
Alternate Flows	5a. Validation Error: System highlights the invalid fields (e.g., missing price) and prevents publication until corrected.

Table 2.5: Use Case 5: Fulfill Order

Field	Description
Use Case ID	UC-05
Use Case Name	Fulfill Order
Primary Actor	Manufacturer
Pre-Conditions	1. Manufacturer is logged in. 2. A confirmed order has been assigned to the Manufacturer.
Post-Conditions	1. Order status is updated to "Shipped". 2. Tracking information is sent to the Customer.
Continued on next page	

Table 2.5 – continued from previous page

Field	Description
Main Success Scenario	1. Manufacturer views the "Pending Orders" queue. 2. Manufacturer selects an order to view production details (files, quantity, size). 3. Manufacturer marks the order as "In Production". 4. System updates the order status visible to the Customer. 5. Manufacturer completes production and packages the item. 6. Manufacturer enters the shipping provider and Tracking ID into the system. 7. Manufacturer marks the order as "Shipped". 8. System triggers a shipment notification email to the Customer.
Alternate Flows	3a. Material Shortage: Manufacturer flags the order as "Delayed" and adds a note. System notifies the Admin to intervene. 6a. Invalid Tracking ID: System rejects the input format and requests a valid ID.

Table 2.6: Use Case 6: Moderate Platform Content

Field	Description
Use Case ID	UC-06
Use Case Name	Moderate Platform Content
Primary Actor	Administrator
Pre-Conditions	1. Administrator is logged in. 2. Content (design/comment) has been flagged by users or automated filters.
Post-Conditions	1. Flagged content is either removed or approved. 2. Action is logged in the moderation history.
Continued on next page	

Table 2.6 – continued from previous page

Field	Description
Main Success Scenario	1. Administrator accesses the "Moderation Queue". 2. System displays a list of reported items with reasons (e.g., copyright, offensive). 3. Administrator selects an item to review details. 4. Administrator evaluates the content against Platform Policies. 5. Administrator selects an action (Dismiss Report, Remove Content, or Ban User). 6. System executes the action and updates the content visibility. 7. System sends an automated notification to the affected user regarding the decision.
Alternate Flows	4a. Ambiguous Case: Administrator escalates the ticket to a "Super Admin" or legal review queue. 5a. User Appeal: If a user appeals a ban, the case re-enters the queue with a "High Priority" tag.

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is organized by modules divided into features as a system feature name and specific functional requirements associated with this feature. This section describes the functional requirements of the system expressed in the natural language style.

2.2.1 Module 1: Client Web Application (React Frontend)

2.2.1.1 Customer Interface

Feature 01: Browse and Search Products

1. The system shall allow customers to view all available products with thumbnails, names, and prices.
2. The system shall allow customers to search for products using keywords, categories, or tags.
3. The system shall display search results dynamically without reloading the page.

4. The system shall provide filters (price range, category, designer, etc.) to refine search results.
5. The system shall sort products by relevance, popularity, or newest first.
6. If no results match the query, the system shall display an appropriate message.

Feature 02: Preview Product

1. The system shall allow customers to select a product and open a detailed view page.
2. The system shall allow customers to preview product variants (e.g., color, size, design).
3. The system shall provide a 3D rendering of the customized product using design preview logic.
4. The system shall notify the user if the product is out of stock.

Feature 03: Place and Pay for Orders

1. The system shall allow customers to add custom products to a shopping cart.
2. The system shall calculate total cost, taxes, and shipping charges automatically.
3. The system shall allow customers to select payment methods (credit/debit card, wallet, etc.).
4. The system shall validate payment information before processing.
5. The system shall include the “Validate Payment” function to ensure secure transaction verification.
6. The system shall confirm order placement and display a summary page after successful payment.
7. The system shall invoke the “Handle Payment Failure” use case if payment is declined or timed out.

Feature 04: Track Order Status

1. The system shall allow users to view the status of all their past and current orders.
2. The system shall display real-time updates (Processing, Shipped, Delivered, Canceled).

3. The system shall include shipment tracking links where available.
4. The system shall notify users automatically when order status changes.

Feature 05: View Personalized Recommendations

1. The system shall display product suggestions based on browsing and purchase history.
2. The system shall update recommendations dynamically as user preferences change.
3. The system shall ensure user privacy when using stored data for recommendations.

2.2.1.2 Designer Interface**Feature 06: Design Canvas**

1. The system shall provide a web-based design workspace with basic drawing, layering, and text editing tools.
2. The system shall support file uploads in PNG, JPEG, and SVG formats.
3. The system shall allow undo/redo actions and autosave progress.
4. The system shall handle upload failures.

Feature 07: Storefront Creation and Management

1. The system shall allow designers to create a personal storefront with name, logo, and description.
2. The system shall allow designers to add or remove products from their storefront.
3. The system shall provide SEO fields for product title, description, and tags.
4. The system shall allow designers to update pricing. (tentative)
5. The system shall notify designers upon successful storefront updates.

Feature 08: Accept Orders

1. The system shall allow designers to view new customer orders and their details.
2. The system shall allow designers to accept or reject custom order requests.
3. The system shall automatically notify customers when an order is accepted or rejected.

Feature 09: Trend Analytics

1. The system shall display analytics for designer storefronts, such as sales, visitors, and most popular items.
2. The system shall generate visual reports for designer performance (includes “Generate Reports”).
3. The system shall notify designers of significant sales milestones achievements.

Feature 10: Product Page SEO Optimization

1. The system shall suggest SEO-friendly titles and tags for designer products using AI analysis or dictionary. (tentative)
2. The system shall allow manual keyword editing and provide real-time feedback.

2.2.1.3 Manufacturer Interface

Feature 11: Accept Orders

1. The system shall display a queue of orders assigned to the manufacturer.
2. The system shall allow manufacturers to view product details, quantity, and deadlines.
3. The system shall allow acceptance or rejection of incoming orders.
4. The system shall log all manufacturer decisions for auditing.

Feature 12: Print and Ship

1. The system shall allow manufacturers to mark orders as printed or shipped.
2. The system shall allow entry of tracking IDs and shipment dates.
3. The system shall automatically trigger notifications to customers upon shipping.
4. The system shall include “Handle Order Fulfillment Delay” when manufacturing issues occur.

Feature 13: Inventory Management

1. The system shall display the available inventory list and current stock levels.
2. The system shall allow adding, editing, or removing inventory items.
3. The system shall alert manufacturers when stock levels fall below predefined thresholds.
4. The system shall extend to “Handle Inventory Shortage usecase” when a requested item is unavailable.

Feature 14: Performance Statistics

1. The system shall display metrics such as average fulfillment time, total orders processed, and defect rates.
2. The system shall allow exporting performance data for administrative reporting.

2.2.1.4 Admin Interface**Feature 15: User Management**

1. The system shall allow the admin to view all registered users.
2. The system shall allow adding, editing, or deactivating user accounts.
3. The system shall restrict sensitive actions to users with admin privileges.
4. The system shall include “Handle Unauthorized Access” for privilege violations.

Feature 16: System Monitoring

1. The system shall display key performance indicators such as uptime, API response time, and transaction volume.
2. The system shall alert the admin of anomalies such as server downtime or failed transactions.

Feature 17: Content Moderation

1. The system shall allow admins to review and remove inappropriate designs or comments.
2. The system shall include “Report Inappropriate Content” triggered by users.
3. The system shall maintain a moderation history log.

Feature 18: Analytics Dashboard

1. The system shall aggregate and visualize platform-wide metrics (sales, traffic, new users).
2. The system shall generate downloadable reports.
3. The system shall include “Handle Report Generation Failure” if data fetching fails.

2.2.2 Module 2: AI and Creative Intelligence

Feature 01: 2D Preview Generator

1. The module shall generate flat 2D mockups of designs on selected merchandise templates.
2. The module shall process designer inputs within 5 seconds for standard resolutions. (still under development, therefore tentative KPI)
3. The module shall notify the frontend if rendering fails.

Feature 02: 3D Renderer

1. The module shall render 3D previews of products using uploaded or generated designs.
2. The module shall allow rotation, and zoom for user visualization.
3. The module shall include “Handle Rendering Failure” for GPU or timeout errors.

Feature 03: Design Suggestion / Generation Engine

1. The module shall generate design ideas based on user prompts or uploaded sketches.
2. The module shall invoke “Handle AI Generation Failure” when inference fails.

Feature 04: Generative Canvas Enhancer

1. The module shall enhance low-resolution designs using up-scaling.
2. The module shall log all enhancement activities for future retraining or audits.(architecture under consideration, therefore tentative)

2.2.3 Module 3: Core Business Logic (Node.js + Express)**Feature 01: Product Service**

1. The service shall manage product creation, updates, and deletion.
2. The service shall validate inputs before saving to the database.

Feature 02: Order Service

1. The service shall orchestrate order creation, tracking, and updates.
2. The service shall notify relevant actors (customer, designer, manufacturer) when order state changes.

Feature 03: Payment Service

1. The service shall manage payment requests, responses, and logs.
2. The service shall communicate securely with the Payment Gateway API.
3. The service shall log all failed transactions for audit and debugging.

Feature 04: Auth Service

1. The service shall authenticate and authorize all API calls.
2. The service shall include “Handle Unauthorized Access” for invalid credentials.

Feature 05: Messaging Service

1. The service shall handle user-to-user and system-to-user communications.
2. The service shall ensure message delivery even under partial failure conditions.

Feature 06: SEO Service

1. The service shall optimize URLs and metadata for better search visibility.

2.2.4 Module 4: Data Management (MongoDB)

1. The module shall create and manage collections for users, products, orders, payments, and logs.
2. The module shall enforce data validation rules and schema consistency.
3. The module shall ensure secure read/write operations using role-based permissions.
4. The module shall handle “Data Sync Failures” by retrying transactions and maintaining logs.

2.2.5 Module 5: Financial Operations (Payment Gateway Integration)

Feature 01: Split Payments

1. The module shall automatically divide payment amounts between designer, manufacturer, and platform commission.

Feature 02: Designer Payouts

1. The module shall initiate scheduled payouts to designer accounts upon reaching the threshold.
2. The module shall handle failed payout retries automatically.

Feature 03: Manufacturer Payouts

1. The module shall calculate and issue payments to manufacturers based on fulfilled orders.
2. The module shall notify the admin upon any transaction errors.

Feature 04: Premium Billing Plan

1. The module shall allow designers to upgrade to premium features.
2. The module shall manage recurring billing and notify users of upcoming renewals.

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements of Orli.

2.3.1 Reliability

- All critical operations (payments, order creation) shall have **automatic retry mechanisms** for failed transactions.
- The system shall store all submitted designs and orders redundantly to prevent loss in case of server restarts.

2.3.2 Usability

- The user interface shall maintain consistent layout and color themes across all pages
- Users shall be able to navigate from homepage to checkout in **no more than 4 clicks**
- Error messages shall be displayed in clear, human-readable language (no system codes)

2.3.3 Performance

- The system shall load the main marketplace page within **5 seconds** on a standard 4G connection
- The system shall handle at least **50 concurrent users** without visible slowdown or request timeout
- The AI-based 2D preview generator shall produce results within **30 seconds**.

2.3.4 Security

- The system shall ensure that cross site scripting will not affect it.
- The system shall ensure that all external integrations, such as AI services or payment gateways, operate over secure, authenticated channels.
- The system shall take measures to prevent SQL injections
- The system shall ensure that all user authentication and authorization processes are performed securely and that only authorized users can access restricted resources.
- The system shall detect and respond to repeated failed login attempts or other suspicious activities in a timely manner.

Chapter 3

System Overview

At its core, the platform empowers designers to build their own brands and storefronts by sourcing materials directly from verified print providers. Designers can create and showcase custom products using integrated design tools, including MS Paint-style interfaces and AI-powered modeling features. These tools allow for real-time 3D rendering enabling customers to visualize products on themselves or in their environment before making a purchase. Below is the architectural visualization of our system focusing on each module separately to ensure all the details are up-front.

3.1 Architectural Design

Orli has been developed using a modular, layered, and service-oriented architecture following the principles of Clean Architecture and Separation of Concerns. It follows the **onion architecture** view from **Head First Design Patterns**.^[?] The system is decomposed into distinct modules, each responsible for a specific set of functionalities. The system, as a whole, follows a multi-layered client-server model, extended with AI integration services and data management subsystems. This decomposition was done based on two concerns:

1. We wanted the core business rules (the "what") independent of the implementation i.e. how it happens.
2. We wanted the modules to evolve or be replaced (e.g., database or AI engine) without impacting the rest of the system.

Hence, the system is divided into five primary modules:

1. client application (presentation layer)

2. core business logic (domain layer)

3. API layer (service layer)

4. creative intelligence layer

5. data management layer

The AI layer and Data Layer are shown as single entity because both come under the guise of persistent storage and store valuable data; one stores user data and other stores AI assets. Each module operates independently but collaborates via well-defined interfaces or APIs. This ensures that the application layer mediates all communication and the core logic remains isolated and intact at all times. Moreover, the interfaces never impact the system implementation.

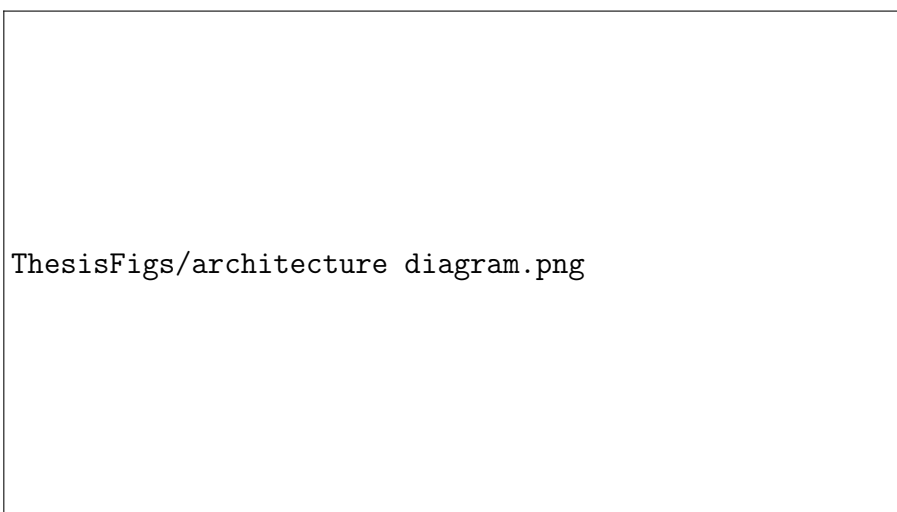


Figure 3.1: System Architecture

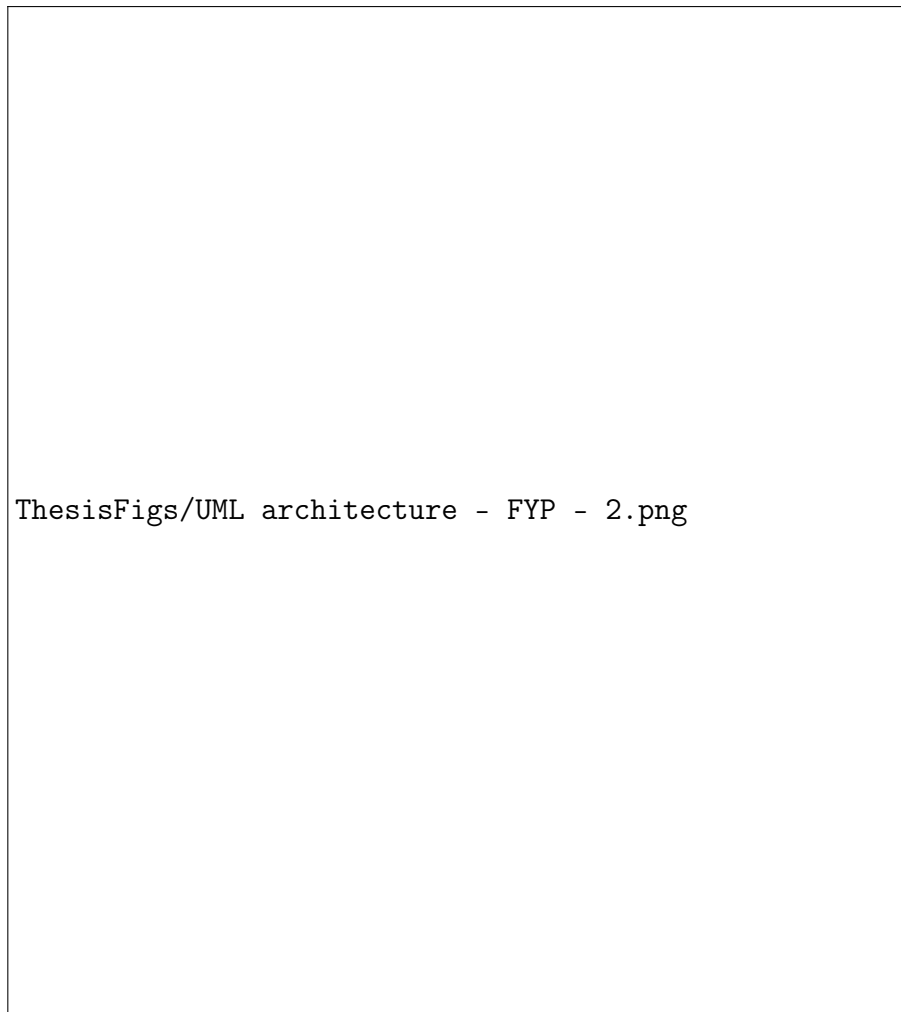


Figure 3.2: System Architecture - Box and Line Diagram

3.2 Data Design

The Orli platform transforms its information domain into structured data entities stored primarily in **MongoDB**, tentatively, following a domain-driven design approach. Each business entity such as *User*, *Design*, *Product*, *Order*, *Storefront*, and *Analytics* directly maps to a database collection.

3.2.0.1 Data Transformation

- **Conceptual to Physical:** Business entities are identified from the domain model, converted into schemas with validation and indexes, and stored as documents for scalability and performance.

3.2.0.2 Major Entities

- **User:** Stores account details, roles (customer, designer, manufacturer, admin), and profile information.
- **Design:** Contains designer-created or AI-generated artwork with metadata and file URLs.
- **Product:** Represents merchandise items linked to designs and manufacturers, including SEO and analytics fields.
- **Order:** Records customer purchases, fulfillment details, and split payment data.
- **Storefront:** Holds designer shop information, themes, and featured products.
- **Analytics and AI Assets:** Capture user engagement metrics and AI processing outputs.
- **Design Assets:** Maintains asset library for Designers

3.2.0.3 Data Storage Systems

- **Database (e.g., MongoDB):** Main database for transactional and relational data.
- **Cloud Storage (e.g., AWS S3):** Stores images, design files, and previews; MongoDB holds only URLs.
- **Cache:** Temporarily stores frequently accessed data for faster performance.
- **AI Model Storage:** Keeps trained AI model versions for design generation.

3.2.0.4 Processing and Organization

Data flows through APIs where user actions (such as uploads or orders) trigger MongoDB updates. Media files are uploaded to cloud storage, while analytics and AI data are aggregated and stored asynchronously.

3.3 Data Design

3.3.1 Transformation of Information Domain

The Orli platform transforms its abstract information domain into structured data entities using a **Domain-Driven Design (DDD)** approach[cite: 958]. This process involves a transition from conceptual modeling to physical schema design:

- **Conceptual Mapping:** Business entities identified during the requirement analysis phase (e.g., Designers, Orders, Products) are mapped directly to distinct database collections[cite: 962].
- **Physical Schema Implementation:** These entities are converted into **MongoDB schemas**, enforcing validation rules and indexing strategies to ensure data integrity and scalability[cite: 964].

3.3.2 Major System Entities

The system's data is organized around the following core entities, which represent the primary business objects[cite: 965]:

- **User:** Stores authentication credentials, profile details, and role-specific attributes for Customers, Designers, Manufacturers, and Administrators[cite: 966].
- **Design:** Contains the creative assets uploaded or generated by designers, including metadata and secure file URLs pointing to cloud storage[cite: 967].
- **Product:** Represents sellable merchandise items. It links a *Design* to a *Manufacturer* and includes SEO tags, pricing, and analytics fields[cite: 968].
- **Order:** Records transaction details, including customer information, fulfillment status, and split-payment calculations for revenue sharing[cite: 969].
- **Storefront:** Holds configuration data for designer shops, including visual themes, descriptions, and featured product lists[cite: 970].
- **Analytics and AI Assets:** specific collections dedicated to storing user engagement metrics and the outputs of AI processing tasks[cite: 971].

3.3.3 Data Storage and Organization

To optimize performance and scalability, the system utilizes a hybrid storage architecture[cite: 973]:

1. **Primary Database (MongoDB):** Acts as the central repository for all transactional and relational data. It stores JSON-like documents for Users, Orders, and Products.

2. **Cloud Storage (Media Management):** Due to the heavy nature of design files and high-resolution product mockups, actual media assets are stored in external cloud services (e.g., Cloudinary or AWS S3). The MongoDB database stores only the *reference URLs* to these files.
3. **Caching Layer:** A temporary storage layer is employed to cache frequently accessed data, reducing database load and improving response times for the market-place interface.
4. **AI Model Storage:** A dedicated storage system is used to maintain versions of trained AI models used for design generation and enhancement.

Processing Flow: Data flows primarily through RESTful APIs. User actions, such as uploading a design, trigger a dual process: the file is uploaded to Cloud Storage, and the resulting URL, along with metadata, is written to MongoDB.

3.4 Domain Model

The domain model is a conceptual representation of the various entities, their attributes, and the relationships between them within the system. It serves as a bridge between the requirements and the design of the software, providing a clear understanding of the business domain and how it interacts with the system being developed.



Figure 3.3: Domain Model

3.5 Design Models

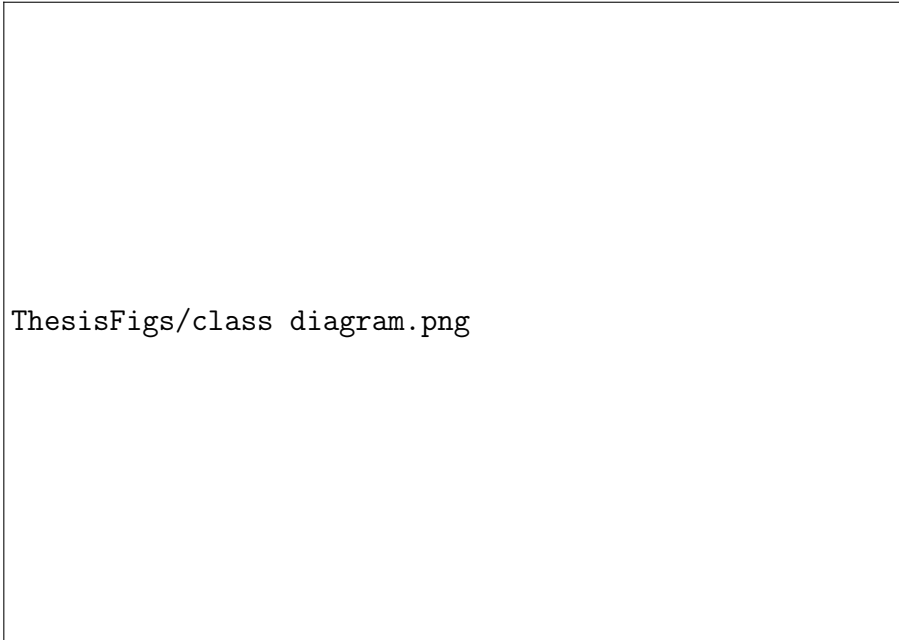


Figure 3.4: Class Digram

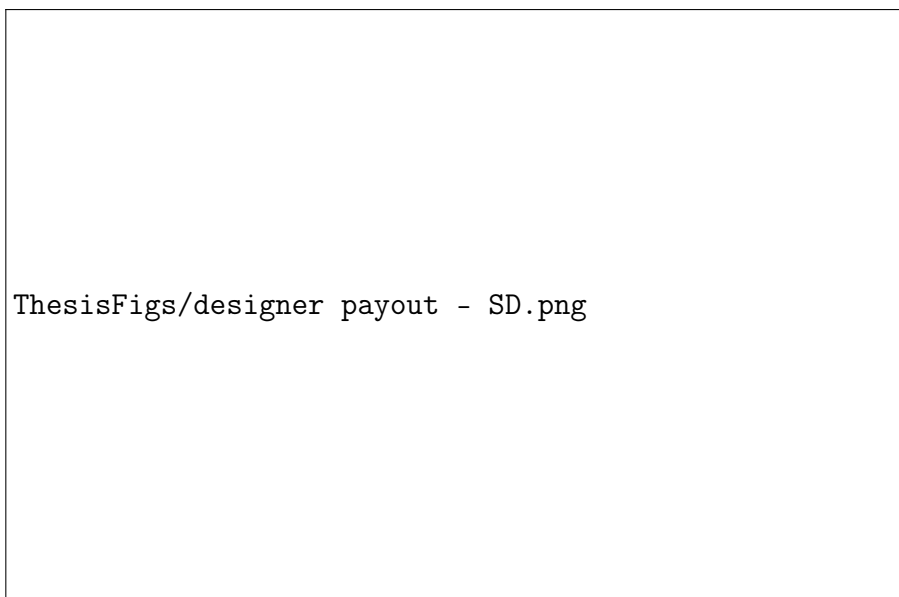
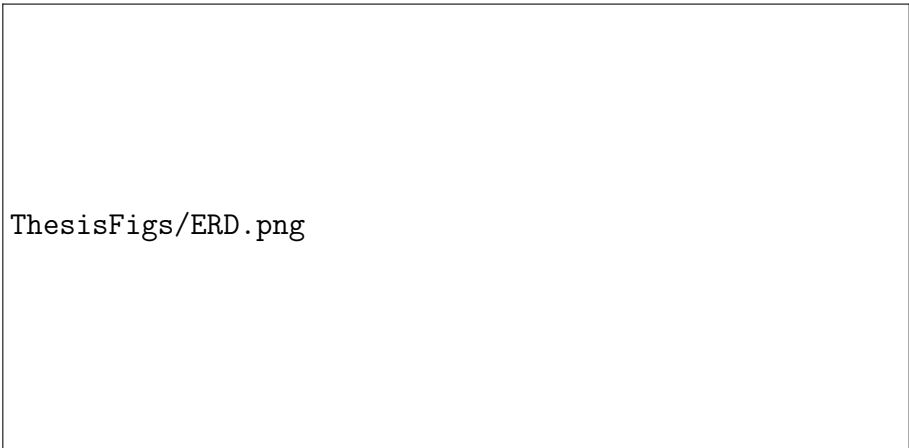


Figure 3.5: DDesigner payout SD



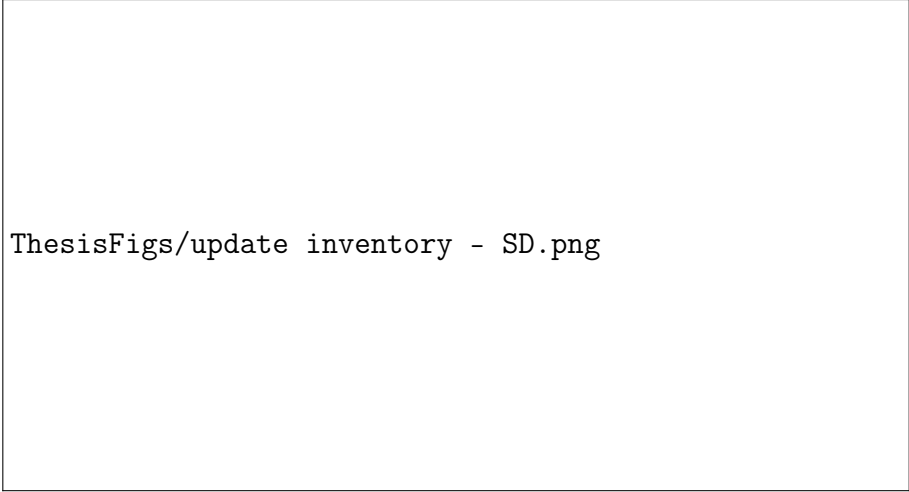
ThesisFigs/designer payout - SD.png

Figure 3.6: DDesigner payout SD



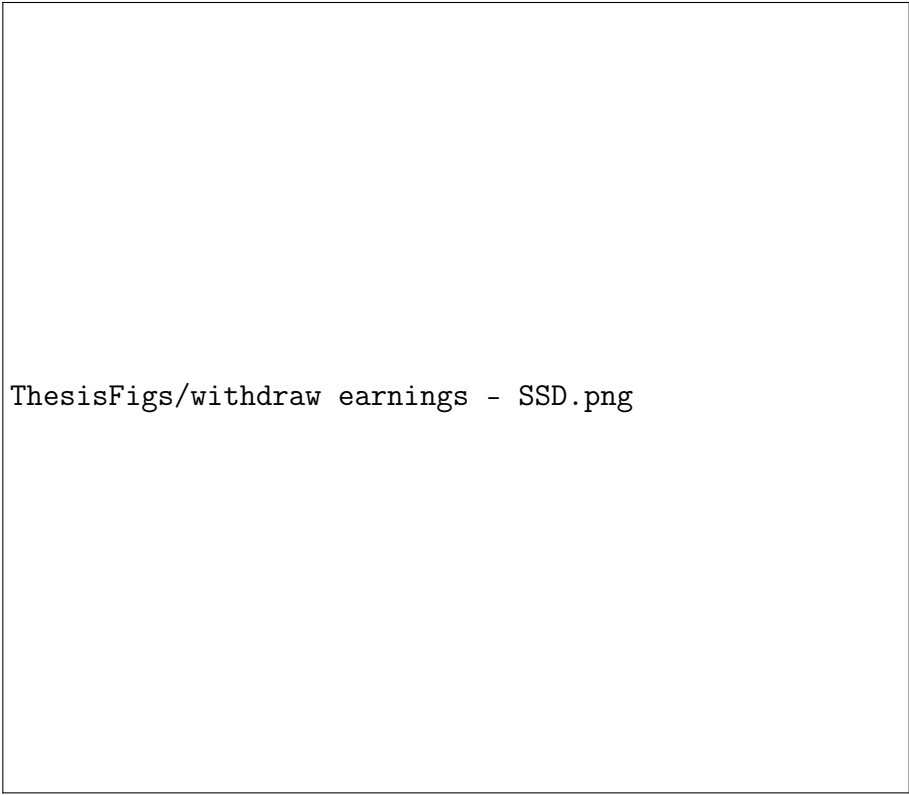
ThesisFigs/ERD.png

Figure 3.7: ERD



ThesisFigs/update inventory - SD.png

Figure 3.8: Update Inventory - SD



ThesisFigs/withdraw earnings - SSD.png

Figure 3.9: Withdraw Earnings - SD



ThesisFigs/manage inventory - SSD.png

Figure 3.10: Manage Inventory - SSD



Figure 3.11: System Activity Diagram



ThesisFigs/manufacturer - activity.png

Figure 3.12: Activity for Manufacturer



Figure 3.13: Activity for Designer

Diagrams are visible on zooming In , please zoom in for clarity

Chapter 4

Implementation and Testing

This chapter outlines the practical implementation of the ORLI platform, focusing on the backend modules for Designers and Manufacturers. The system is built using the MERN stack (MongoDB, Express, React, Node.js) with Python-based AI services detailed in previous chapters.

ORLI aims to democratize print-on-demand by integrating AI-driven design tools directly into the marketplace. The context of this iteration is the core commerce flow: enabling designers to create value (products) and manufacturers to fulfill that value (orders).

4.1 Algorithm Design

To ensure robustness in financial and logistical operations, specific algorithms were designed for the core modules. Below are the pseudocode explanations for the Designer Payout system and Inventory Management.

4.2 External APIs/SDKs

The following table describes the third-party APIs and SDKs used in the project implementation.

Algorithm 1 Calculate Designer Payouts

```
1: Input: Today's Date
2: Output: List of Processed Payouts
3: threshold  $\leftarrow$  50.00
4: designers  $\leftarrow$  DB.Users.find({role : "DESIGNER"})
5: for each designer in designers do
6:   pending_earnings  $\leftarrow$  0
7:   orders  $\leftarrow$  DB.Orders.find({designerId : designer.id, status : "DELIVERED", payout : "UNPAID"})
8:   for each order in orders do
9:     pending_earnings  $\leftarrow$  pending_earnings + (order.price  $\times$  designer.royalty)
10:  end for
11:  if pending_earnings  $\geq$  threshold then
12:    payout  $\leftarrow$  CreatePayout(designer.id, pending_earnings)
13:    response  $\leftarrow$  PaymentGateway.transfer(payout)
14:    if response.success then
15:      DB.Payouts.save(payout)
16:      NotifyDesigner(designer.id, "PayoutSuccess")
17:    end if
18:  end if
19: end for
```

Algorithm 2 Inventory Allocation (Optimistic Locking)

```
1: Input: OrderItems List
2: Output: Boolean (Success/Failure)
3: Transaction.Start()
4: for each item in OrderItems do
5:   variant  $\leftarrow$  DB.ProductVariants.findById(item.variantId)
6:   if variant.stockLevel < item.quantity then
7:     Transaction.Rollback()
8:     return False, "Insufficient Stock"
9:   end if
10:  update  $\leftarrow$  DB.ProductVariants.update({_id : variant.id}, { $inc : {stock : -item.quantity} })
11:  if update.modifiedCount == 0 then
12:    Transaction.Rollback()
13:    return False, "Concurrency Error"
14:  end if
15: end for
16: Transaction.Commit()
17: return True, "Allocated"
```

API and version	Description	Purpose of usage	API endpoint/function used
Stripe (v2022-11)	International Payments	Credit Card processing for global customers	stripe.paymentIntents.create
JazzCash (Sandbox)	Local Payment Gateway	Mobile Wallet payments (Pakistan)	jazzcash.api.createMobilePayment
EasyPaisa (Sandbox)	Local Payment Gateway	Mobile Wallet payments (Pakistan)	easypaisa.checkout.initiate
SadaPay (Sandbox)	Local Debit Gateway	Fee-free debit card transactions	sadapay.charges.create
Cloudinary (v2)	Media Management	Uploading Product Images	cloudinary.uploader.upload
SendGrid (v3)	Email Service	Sending transactional emails	sgMail.send

Table 4.1: Third-Party APIs and SDKs

Bibliography